SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Shaik Basheer	**Reg. No.:**	192465049
Section / Batch:	CSE – AI /2024	**Date:**	22-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

E-Learning Content Recommendation Optimization (Graph + Ranking)

An e-learning platform must recommend personalized content to students based on learning patterns. Analyse how graph-based ranking algorithms can be applied. Identify constraints such as user diversity and content volume. Design an optimized recommendation system. Discuss scalability in large educational platforms. Evaluate computational efficiency. Interpret results in terms of learning effectiveness.

1. Introduction

An e-learning platform contains a large number of courses, videos, quizzes, assignments, and learning materials. Different students have different learning patterns, interests, skill levels, and learning speeds.

A recommendation system can use graph-based ranking algorithms to identify relationships between students and learning content and recommend the most suitable content.

In this system, students are represented as user nodes, learning resources as content nodes, and interactions such as viewing, completing, liking, or rating as edges. A ranking algorithm calculates the relevance score of each content item and recommends the highest-ranked content.

The main goal is to provide personalized recommendations while maintaining computational efficiency and scalability.

2. Understanding of the Industry Problem

The e-learning platform faces several major problems:

1. User diversity – Students have different backgrounds, interests, knowledge levels, and learning speeds.
2. Large content volume – Thousands or millions of videos, courses, quizzes, and documents may be available.
3. Different learning patterns – Some students prefer videos, while others prefer reading or practical exercises.
4. Sparse interaction data – A new student may have very few interactions with the platform.
5. Changing interests – A student's learning requirements may change as they progress.
6. Real-time recommendation requirement – Recommendations should be generated quickly when the student accesses the platform.

Therefore, the platform needs an optimized recommendation system that can rank relevant content efficiently.

3. Application of Graph-Based Ranking

The recommendation system can represent the e-learning environment as a graph.

Students, courses, videos, quizzes, topics, and skills can be represented as nodes. Relationships such as watched, completed, attempted, belongs to, related to, and teaches can be represented as edges.

Example:

Student A → watched → Python Course → related to → Python Quiz → related to → Data Structures

A graph-based ranking algorithm can use these connections to calculate how relevant each learning resource is to a particular student.

4. Graph-Based Ranking Method

A recommendation score can be assigned to every content item.

$Score(u,i) = w1R + w2P + w3S + w4C$

Where:

R = relevance to the student's interests

P = previous interaction score

S = similarity with completed content

C = content quality score

w1, w2, w3, w4 = weights

The system ranks the content according to the final score and recommends the highest-ranked items.

5. Proposed Recommendation System

The proposed system follows this workflow:

Student Activity

↓

Collect Learning Data

↓

Construct User-Content Graph

↓

Generate Candidate Content

↓

Calculate Graph-Based Scores

↓

Apply Ranking

↓

Top-K Recommendations

↓

Student Learning

↓

Collect New Feedback

↓

Update Recommendations

Step 1: Collect learning data such as videos watched, courses completed, quiz scores, time spent learning, search history, and ratings.

Step 2: Construct a graph where students, content, topics, and skills are nodes and their interactions are edges.

Step 3: Generate candidate content instead of ranking the entire content database.

Step 4: Calculate a relevance score for each candidate.

Step 5: Select the top-K content items.

Step 6: Update the graph using new student interactions.

6. Constraints

User Diversity:

Students have different skill levels, learning interests, previous knowledge, learning speeds, and preferred content formats.

Content Volume:

Large e-learning platforms may contain a huge number of resources. Ranking every item for every student can require significant computation.

Cold-Start Problem:

A new student may not have enough interaction history. Newly added content may also have very little interaction data.

Dynamic Learning Patterns:

Student preferences can change over time, so the recommendation system must continuously update its rankings.

7. Optimization Techniques

1. Candidate Filtering – Select relevant content before detailed ranking.
2. Top-K Ranking – Return only the highest-ranked items.
3. Graph Partitioning – Divide the graph based on subjects, topics, or skill levels.
4. Caching – Store frequently requested recommendations.
5. Incremental Updates – Update only the affected part of the graph instead of rebuilding it completely.

8. Suitable Tools and Technologies

Python – Recommendation algorithm development

NetworkX – Graph construction and analysis

Pandas – Learning-data processing

NumPy – Numerical computation

Scikit-learn – Similarity and ranking support

Neo4j – Large graph database

VS Code – Development and debugging

Git/GitHub – Version control

Apache Spark – Large-scale data processing

For an academic implementation, Python, NetworkX, Pandas, and Scikit-learn are sufficient. For a large educational platform, a graph database and distributed processing technologies can be considered.

9. Sample Algorithm

Algorithm: Graph-Based Content Recommendation

Input:

Student u

User-content graph G

Content set C

Number of recommendations K

Step 1: Get the student's learning history.

Step 2: Find content connected to previous learning activities.

Step 3: Generate candidate content.

Step 4: Calculate relevance score for every candidate.

Step 5: Sort candidates according to their scores.

Step 6: Select the top K content items.

Step 7: Display recommendations to the student.

Step 8: Update the graph using new student interactions.

Output:

Top-K personalized learning recommendations.

10. Computational Efficiency

If U represents the number of users, C the number of content items, and E the number of graph relationships, ranking every content item for every user can become expensive.

Therefore, the proposed system uses:

Candidate Generation → Ranking → Top-K Selection

For example:

Total Content = 100,000

↓

Candidate Filtering

↓

Candidates = 1,000

↓

Ranking

↓

Top 10 Recommendations

This approach reduces unnecessary computation. The exact computational complexity depends on the selected graph-ranking algorithm, graph structure, candidate-generation method, and implementation.

11. Scalability in Large Educational Platforms

For a large platform with millions of users and content items, scalability can be improved using:

1. Distributed processing – Divide recommendation calculations across multiple machines.
2. Graph databases – Efficiently store and retrieve relationships.
3. Caching – Store frequently requested recommendations.
4. Parallel ranking – Rank different candidate groups simultaneously.
5. Partitioning – Divide the graph into smaller sections.
6. Incremental processing – Update only changed relationships.
7. Top-K recommendation – Avoid ranking unnecessary content.

12. Performance Evaluation

The recommendation system can be evaluated using:

• Recommendation response time

• CPU usage

• Memory consumption

• Number of users supported

• Number of content items processed

• Recommendation accuracy

• Precision@K

• Recall@K

• Click-through rate

• Course completion rate

• Student engagement

• Learning improvement

Example evaluation:

Metric | Before Optimization | After Optimization

Candidate Items | 100,000 | 1,000

Ranking Time | High | Lower

Recommendations | Generic | Personalized

Memory Usage | High | Reduced

Response Time | High | Lower

The exact numerical values should be obtained from actual testing rather than assumed.

13. Result Interpretation

The graph-based recommendation system can identify relationships between students and learning resources.

For example, if a student frequently studies Python, the graph can identify related topics such as Functions, Data Structures, Algorithms, and Programming Projects. The system can therefore recommend content that follows the student's learning path. A successful system should result in more relevant learning content, reduced search time, increased student engagement, better personalized learning, improved course completion, and more efficient use of computational resources. Recommendation quality should ultimately be validated using real student interaction and learning-outcome data.

14. Advantages

1. Provides personalized recommendations.
2. Uses relationships between students and content.
3. Can consider multiple types of learning interactions.
4. Supports Top-K recommendations.
5. Can be optimized for large datasets.
6. Can continuously update recommendations.
7. Helps students discover relevant learning materials.

15. Limitations

1. Large graphs require significant memory and processing.
2. New students may have insufficient interaction data.
3. New content may initially have low ranking information.
4. Frequent graph updates can increase computational cost.
5. Poor-quality interaction data can reduce recommendation quality.
6. Very large platforms require distributed and scalable infrastructure.

16. Conclusion

The proposed graph-based recommendation system provides an effective method for personalized e-learning content recommendation. By representing students, learning resources, topics, and skills as nodes and their relationships as edges, the system can identify relevant content and rank it according to student learning patterns.

Candidate filtering, Top-K ranking, caching, graph partitioning, and incremental updates can improve computational efficiency and scalability.

For large educational platforms, scalable graph storage and distributed processing can help handle millions of students and learning resources. The final effectiveness should be evaluated using recommendation accuracy, response time, student engagement, course completion, and actual learning outcomes.